

Lab 3: Motion Planning with a 6-DOF Manipulator

(72 points total)

1 Main Implementation

In this lab assignment, you will implement the RRT algorithm for a 6-DOF robotic manipulator. To perform the assignment, you will need to have installed the AIKIDO infrastructure. We provide for you the file `adarrt.py`, which contains several methods you will fill in. During development, you will run your code in simulation with -

```
$ python adarrt.py --sim
```

The python file contains the following classes and functions:

- `AdaRRT`: This is the main class. It initializes the start and goal node, the number of iterations, the step_size δ when extending a node in the tree, the desired goal precision ϵ and the joint limits of the robot. It also includes information about the environment, which includes a table, a soda can that we want to grasp, the robot and a set of constraints that check for collisions. This implementation will be very similar to the RRT you built in HW4, with a few modifications discussed below.
- `AdaRRT.Node`: A Node object should contain a copy of the state, a pointer to the parent node in the tree, and a list of pointers to all its child nodes in the tree. A state in the provided code is a 6D `np.array` that contains the robot's configuration.
- `main`: this function specifies the start and goal configurations, sets up the RRT planner and computes a path. It then calls the AIKIDO function `compute_joint_space_path`, which generates a trajectory for the robot to follow.

Steps to complete the lab:

1. **Implement an RRT algorithm** by filling in the code in the provided file. For start-

ing configuration q_S and goal configuration q_G , and parameters ϵ and δ use:

$$\begin{aligned} q_S &= [-1.5, 3.22, 1.23, -2.19, 1.8, 1.2] \\ q_G &= [-1.72, 4.44, 2.02, -2.04, 2.66, 1.39] \\ \delta &= 0.25 \\ \epsilon &= 1.0 \end{aligned}$$

Make sure you have roscore running before starting your RRT!

2. **Visualize the trajectory in rviz.** First, execute your AdaRRT implementation, but don't execute the trajectory. Then, open rviz from the command line using `roslaunch rviz rviz`. In the bottom left module, click the "Add" button and navigate to the "By Topic" tab. You should see a `InteractiveMarkers` topic under `/dart_markers`. Add the topic before executing your trajectory generated by AdaRRT.

- (a) Accept the GitHub Classroom invitation
(GitHub Classroom)
- (b) Download the docker image (we will not use the same docker images as in Lab1 and Lab2).

If use **ARM-64** architecture:

- If use Python2:
Download and directly load this docker image:
(Google Drive)
`docker load -i cs545-lab3-melodic-arm64.tar.`
- If use Python3:
Download and directly load this docker image:
(Google Drive)
`docker load -i cs545-lab3-noetic-arm64.tar.`

If use **X86-64** architecture:

- If use Python2:
Download and directly load this docker image:
(Google Drive)
`docker load -i cs545-lab3-melodic-x86-64.tar.`

If build your own Docker image:

- Follow this instruction:
(Google Drive)

- (c) Build the provided workspace (already in the docker image, you can skip this step): (Google Drive)

Unzip it to your home directory. Before running the assignment codes, make sure you have run `source /ros_ws/devel/setup.bash` in the same terminal as your lab script. Remove build, devel, and log from your ros_ws and rebuild the workspace by running

```
catkin_make_isolated -j4 # Use 4 parallel jobs
```

(d) Create Docker Container

(e) Run lab3 code

If use **ARM-64** architecture:

- Terminal 1: Run roscore
Run roscore
`source /opt/ros/melodic/setup.bash`
`roscore`
- Terminal 2: Run your script
Put your script in ' /ros_ws/src/lab3/adarrt.py'.
Run your script:
`source ~/ros_ws/devel_isolated/libada/setup.bash`
`export ROS_PACKAGE_PATH=$ROS_PACKAGE_PATH:/root/ros_ws/src/libada`
`python ~/ros_ws/src/lab3/adarrt.py --sim`
- Terminal 3: Run Rviz visualization
`source /opt/ros/melodic/setup.bash`
`source ~/ros_ws/devel_isolated/libada/setup.bash`
`export ROS_PACKAGE_PATH=$ROS_PACKAGE_PATH:/root/ros_ws/src/libada`
`roslaunch rviz rviz`

If use **X86-64** architecture:

- Terminal 1: Run roscore
Run roscore
`source ~/.bashrc`
`roscore`
- Terminal 2: Run your script
Put your script in ~/lab3/adarrt.py.
Run your script:
`source ~/.bashrc`
`python ~/lab3/adarrt.py --sim`
- Terminal 3: Run Rviz visualization
`source ~/.bashrc`
`roslaunch rviz rviz`

3. **(40 points)** Use an off-shelf screen capture software (e.g., <https://itsfoss.com/kazam-screen-recorder/>) to **record a video** of the trajectory. Include the video in the root of your GitHub repo as a file named question-3.mp4.

4. **(10 points)** The RRT trajectory is typically jerky. Typical planners use shortcutting algorithms to make the path smoother. **Replace the function** `ada.compute_joint_space_path` with `ada.compute_smooth_joint_space_path`. Capture the new trajectory with two videos – one showing the default isometric view, and another showing the top view. Include the videos in the root of your GitHub repo as files named `question-4-default.mp4` and `question-4-top.mp4`.
5. **(10 points)** The goal precision ϵ of 1.0 in the previous question is too large. In order to avoid collisions, we need to improve the precision. However, this dramatically increases the time to compute a solution. To improve computation, **add a method** `_get_random_sample_near_goal` that generates a sample around the goal within a distance of 0.05 along each axis of the search space. Then, change the build method so that it calls `_get_random_sample_near_goal` with probability 0.2 and `_get_random_sample` with probability 0.8. Reduce ϵ to 0.2.

Write down your observations in the PDF file. Also capture the new trajectory with two videos – one showing the default isometric view, and another showing the top view. Include the videos in the root of your GitHub repo as files named `question-5-default.mp4` and `question-5-top.mp4`.

Answer:

From the observation, the time spent on finding the trajectory in question 5 is relatively shorter than that in question 4. It can be seen from figure 1 that question 5 takes around 3.6 seconds, while for question 4, it requires more than 3.6 seconds. This could be because of the use of `_get_random_sample_near_goal` function, which increases the probability that RRT expands towards the goal position, thus reducing the time to find a path, while the effect of reducing the goal precision is less obvious. Meanwhile, the path waypoints in question 5 are fewer compared with that in q4, which reflects that the path of q5 appears smoother than that of q4 after shortcutting.

Although it is hard to observe the effect of the reduced goal precision, q5 helps the search progress toward the target more consistently, which may contribute to reaching the position closer to the target compared with q4, reducing the final pose error.

In addition, if there are obstacles, the `_get_random_sample_near_goal` function will improve the success rate compared with just random sampling in q4, because it can maintain exploration using random sampling and bias the sampling process toward the goal as well.

```
Warning [CatkinResourceRetriever.cpp:228] [CatkinResourceRetriever::getWorkspaces] Failed reading package
source directories from the marker file '/usr/local/.catkin'. Resources in the source space of this work
space will not resolve.
Path waypoints:
[[-1.5      3.22      1.23      -2.19      1.8      1.2      ]
 [-1.53787831 3.39604058 1.33847517 -2.16823061 1.93134865 1.22411868]
 [-1.57108084 3.5701318 1.4548471 -2.14029793 2.05603839 1.25904212]
 [-1.60830373 3.74864104 1.57194963 -2.1288059 2.17853465 1.27900479]
 [-1.72      4.44      2.02      -2.04      2.66      1.39      ]]
3.604421seconds elapsed
Press ENTER to execute trajectory and exit
```

(a) Waypoints and running time of default view of q5

```
Warning [CatkinResourceRetriever.cpp:228] [CatkinResourceRetriever::getWorkspaces] Failed reading package
source directories from the marker file '/usr/local/.catkin'. Resources in the source space of this work
space will not resolve.
Path waypoints:
[[-1.5      3.22      1.23      -2.19      1.8      1.2      ]
 [-1.4657457 3.2049742 1.4310323 -2.0633502 1.82723693 1.2624889 ]
 [-1.28032945 3.29761073 1.53613869 -2.13745707 1.79891481 1.30937133]
 [-1.07628027 3.36678245 1.53624244 -2.03811951 1.8753833 1.3284452 ]
 [-1.18052421 3.53974774 1.60589188 -2.03655725 2.0046715 1.34058137]
 [-1.28256625 3.71525205 1.67938463 -2.04106612 2.13055808 1.34466967]
 [-1.38759548 3.88771656 1.76625871 -2.04730206 2.24939143 1.34885258]
 [-1.72      4.44      2.02      -2.04      2.66      1.39      ]]
3.55235seconds elapsed
Press ENTER to execute trajectory and exit
```

(b) Waypoints and running time of top view of q5

```
Warning [CatkinResourceRetriever.cpp:228] [CatkinResourceRetriever::getWorkspaces] Failed reading package
source directories from the marker file '/usr/local/.catkin'. Resources in the source space of this work
space will not resolve.
Path waypoints:
[[-1.5      3.22      1.23      -2.19      1.8      1.2      ]
 [-1.27579455 3.25587062 1.29035898 -2.20152615 1.88253329 1.21890532]
 [-1.21029757 3.29143123 1.41975226 -2.01024094 1.83545887 1.18152977]
 [-1.00793046 3.25966615 1.48533552 -1.88491676 1.85639611 1.19116584]
 [-0.82450128 3.32852739 1.61673006 -1.9514053 1.84834491 1.23976413]
 [-0.73410957 3.49041683 1.62030569 -2.03824864 1.97681116 1.30350598]
 [-0.62671535 3.59409186 1.57872137 -1.88892168 1.87735979 1.38287598]
 [-0.4897632 3.68212107 1.63970826 -1.80329706 1.91832153 1.53540836]
 [-0.32590939 3.76986375 1.56172142 -1.67162391 1.87086376 1.58316971]
 [-0.25722942 3.82357802 1.45715815 -1.82970608 1.83338132 1.71571954]
 [-0.30304037 3.92738593 1.58878944 -1.94638356 1.82972703 1.8523632 ]
 [-0.27343318 3.93619391 1.77479535 -2.0298892 1.69996654 1.90836907]
 [-0.28733659 3.89495356 1.94997771 -2.11625389 1.81574764 2.00351603]
 [-0.37046256 4.02597973 2.03945967 -2.20969009 1.74767897 1.87293454]
 [-0.47803752 3.9979494 2.19580761 -2.20921118 1.87177539 1.97440916]
 [-0.61757524 4.13221936 2.30837553 -2.20171281 1.93943645 1.88668789]
 [-0.77533372 4.21783415 2.41081254 -2.11617821 2.05021413 1.87250461]
 [-0.95683534 4.28193691 2.53423906 -2.11738638 2.14251424 1.83135705]
 [-0.83905088 4.33620364 2.48661313 -2.08348013 2.34807586 1.8284413 ]
 [-0.96741479 4.235108 2.35380275 -2.09376018 2.4756078 1.87079349]
 [-1.72      4.44      2.02      -2.04      2.66      1.39      ]]
3.722715seconds elapsed
Press ENTER to execute trajectory and exit
```

(c) Waypoints and running time of default view of q4

```
Warning [CatkinResourceRetriever.cpp:228] [CatkinResourceRetriever::getWorkspaces] Failed reading package
source directories from the marker file '/usr/local/.catkin'. Resources in the source space of this work
space will not resolve.
Path waypoints:
[[-1.5      3.22      1.23      -2.19      1.8      1.2      ]
 [-1.35629971 3.3278872 1.36961325 -2.13753083 1.86702017 1.25894066]
 [-1.15821657 3.39107529 1.47046972 -2.07183906 1.87256233 1.32787675]
 [-1.18035224 3.39753914 1.49068794 -1.83658488 1.81331145 1.37987924]
 [-1.16405376 3.56268784 1.39424088 -1.77918826 1.83695631 1.23221482]
 [-1.19703927 3.70983792 1.4121402 -1.68202219 1.66393719 1.24006121]
 [-1.24293281 3.88157395 1.33034559 -1.53484787 1.64649195 1.28744837]
 [-1.33274704 4.06435303 1.30408887 -1.52829811 1.75778341 1.37637031]
 [-1.41690829 4.18654048 1.38234008 -1.35592673 1.80124025 1.42893901]
 [-1.58274558 4.26371341 1.39390538 -1.33224058 1.95852416 1.48901659]
 [-1.73280427 4.35433936 1.30644071 -1.32201818 2.09280944 1.41167178]
 [-1.82238283 4.41701611 1.4116206 -1.50456301 2.14485392 1.35290737]
 [-1.72      4.44      2.02      -2.04      2.66      1.39      ]]
3.646132seconds elapsed
Press ENTER to execute trajectory and exit
```

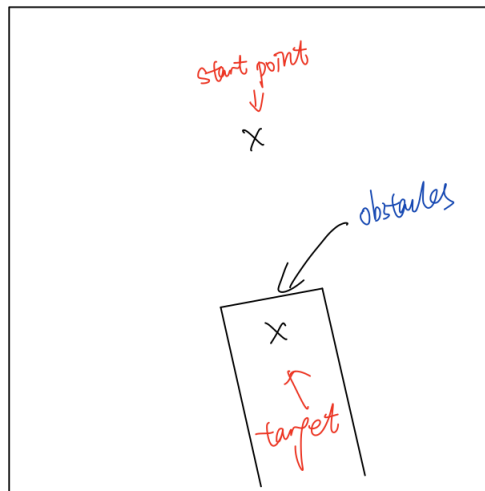
(d) Waypoints and running time of top view of q4

Figure 1: Waypoints and running time of q4 and q5

6. **(10 points)** Explain why it is not a good idea to call `_get_random_sample_near_goal` with probability 1.0. Also present an example where this could be problematic. Write your answer in the PDF.

Answer:

Because if we set `get_random_sample_near_goal` with probability 1.0, it will simply grow the tree directly towards the goal with no exploration of other possible paths in the configuration space. This probability setting creates problem when we have a U-shape obstacle as the graph shows:



It will struggle to find a true path if we set the probability to be 1.0 for the `get_random_sample_near_goal` function since the robot will continuously collide with the obstacle and fail to find a solution.

In-person lab: Once you are confident in your simulation results, you are ready to run it on the real robot. This can be done on the lab workstations with -

```
$ python adarrrt.py --real
```

Refer to Piazza for more instructions on scheduling time in the lab.

2 Additional Questions

As a very rough guideline, we anticipate that for most teams, the answers to each question below will be approximately one paragraph long (or about 4-5 bullet points). How-

ever, your answers may be shorter or longer if you believe it is necessary.

2.1 Resources Consulted

Question: (1 points) Please describe which resources you used while working on the assignment. You do not need to cite anything directly part of the class (e.g., a lecture, the CSCI 545 course staff, or the readings from a particular lecture). Some examples of things that could be applicable to cite here are: (1) did you get help from a classmate *not* part of your lab team; (2) did you use resources like Wikipedia, StackExchange, or Google Bard in any capacity; (3) did you use someone's code (again, for someone *not* part of your lab team)? When you write your answers, explain not only the resources you used but HOW you used them. If you believe your team did not use anything worth citing, *you must still state that in your answer* to get full credit.

Answer:

We searched for a screen recording tool, since initially the kazam did not work properly, as it only recorded the mouse pointer and not the full screen. After searching, we used OBS studio to record the videos.

Meanwhile, we used Format Factory software to edit the videos, deleting redundant content before the simulator execution.

In addition, We asked ChatGPT to generate the LaTeX figure format.

2.2 Team Contributions

Question: (1 points) Please describe below the contributions for each team member to the overall lab. *Furthermore, state a (rough) percentage contribution for each member.* For example, in a team of 4, did each team member contribute roughly 25% to the overall effort for the project?

Answer:

Basically everyone in our group worked on the full implementation of this lab and we compared our codes and results in a self-organized group meeting. We have pushed Zixuan Zhu's adarrt.py to github for grading. In terms of the report, we have Grace Xu in charge of Q3, Jefferson Wang in charge of Q4, Zixuan Zhu in charge of Q5 and Han Zhang in charge of Q6. We want to give an equal 25% contribution to each individual.